算法小册 - analysis in Go

- 第一章 数论
  - 1.1 GCD
  - 1.2 快速幂
  - 1.3 素数线性筛
  - 1.4 概率期望
- 第二章 排序
  - 2.1 冒泡排序
  - 2.2 快速排序
  - 2.3 归并排序
  - 2.4 堆排序
  - 2.5 无序数组中位数
  - 2.6 求两个有序数组的中位数或者第k小元素
  - 2.7 全排列
- 第三章 数据结构
  - 3.1 链表
    - 求单链表中结点的个数
    - 将单链表反转
    - 查找单链表中的倒数第K个结点（k > 0）
    - 查找单链表的中间结点
    - 从尾到头打印单链表
    - 合并两个排序的链表
    - 判断一个单链表中是否有环
    - 两个单链表相交的第一个节点
    - 已知单链表中存在环，求进入环中第一个节点
    - 删除链表的节点
  - 3.2 队列
  - 3.3 栈
  - 3.4 堆
  - 3.5 二叉树
    - 二叉搜索树的最近公共祖先
    - 二叉树的直径
    - 二叉树的最大距离（即相距最远的两个叶子节点）
    - 二叉树的深度
    - 二叉树的宽度
  - 3.6 并查集
  - 3.7 树状数组
  - 3.8 线段树
  - 3.9 Trie树
  - 3.10 散列表
  - 3.11 二叉排序树
- 第四章 动态规划
  - 4.1 01背包问题
  - 4.2 最长递增子序列
  - 4.3 最长公共子序列
  - 4.4 最大字段和
  - 4.5 树塔问题
  - 4.6 两个字符串的编辑距离
- 第五章 搜索
  - 5.1 二分查找
  - 5.2 BFS
  - 5.3 DFS
  - 5.4 A*
  - 5.5 三分

第六章 图算法
6.1 最短路径
6.2 二分图
6.3 拓扑排序
6.4 最小生成树
第七章 字符串算法
7.1 KMP
7.2 BM
7.3 Manacher
7.4 AC自动机
第八章 海量数据处理
8.1 布隆过滤器
8.2 Bitmap
8.3 Simhash
8.4 MapReduce

第一章 数论

1.1 GCD

```
package main
import "fmt"

func gcd(a ,b int) int {
    for {
        if b == 0 {
            return a
        }
        c := a%b
        a = b
        b = c
    }
}

func main () {
    fmt.Println(gcd(15, 12))
}
```

1.2 快速幂

```
package main
import "fmt"

func qpow(x, n int) int {
    if n == 0 {
        return 1
    }
    if n < 0 {
        x = 1 / x
        n = -1 * n
    }
```

```
    result := 1
    for {
        if n == 0 {
            break
        }
        if n&1 == 1 {
            result *= x
        }
        x *= x
        n >>= 1
    }

    return result
}

func main () {
    fmt.Println(qpow(2, 10))
}
```

1.3 素数线性筛

```
package main
import "fmt"

func findPrimes(n int) []int {
    var results []int
    isPrimeMap := make(map[int]bool)
    isPrimeMap[0], isPrimeMap[1] = false, false
    for i := 2; i<=n; i++ {
        if _, ok := isPrimeMap[i]; !ok{
            results = append(results, i)
        }
        for j:=i; j<=n; j+=i {
            isPrimeMap[j] = false
        }
    }
    return results
}

func main () {
    fmt.Println(findPrimes(100))
}
```

1.4 概率期望

第二章 排序

2.1 冒泡排序

```
package main
import "fmt"

func max(a, b int) int {
    if a > b {
        return a
```

```go
    }
    return b
}

func bubbleSort(a[]int) []int {
    n := len(a)
    for i:=n-1; i>=0; i-- {
        for j:=0; j<i; j++ {
            if a[i] < a[j] {
                a[i], a[j] = a[j], a[i]
            }
        }
    }
    return a
}

func main () {
    a := []int{1, 3, 2, 5, 3, 7, 4, 5}
    fmt.Println(bubbleSort(a))
}
```

2.2 快速排序

```go
package main

import "fmt"

func quickSort(a []int, l int , r int) {
    if l >= r {
        return
    }
    x := a[l]
    i, j := l, r
    for i < j {
        for i<j && a[j]>=x {
            j-=1
        }
        if i < j {
            a[i] = a[j]
            i+=1
        }

        for i<j && a[i]<x {
            i+=1
        }
        if i < j {
            a[j] = a[i]
            j-=1
        }
    }
    a[i] = x
    quickSort(a, l, i-1)
    quickSort(a, i+1, r)
}

func main() {
    a := []int{5, 4, 3, 2, 1}
```

```
    quickSort(a,0, len(a)-1)
    fmt.Println(a)
    return
}
```

2.3 归并排序

2.4 堆排序

2.5 无序数组中位数

2.6 求两个有序数组的中位数或者第k小元素

2.7 全排列

第三章 数据结构

3.1 链表

链表结构

```
type ListNode struct {
    Val int
    Next *ListNode
}
```

求单链表中结点的个数

```
fun GetListLen(head *ListNode) int {
    var result 0
    for {
        if head == nil {
            break
        }
        head = head.Next
        result += 1
    }
    return result
}
```

将单链表反转

```
func reverseList(head *ListNode) *ListNode {
    if head == nil || head.Next == nil {
        return head
    }

    next := head.Next
    head.Next = nil

    for {
        if next.Next == nil {
```

```
            break
        }
        nextNext := next.Next

        next.Next = head
        head = next
        next = nextNext
    }
    next.Next = head

    return next
}
```

查找单链表中的倒数第K个结点（k > 0）

```
func getKthFromEnd(head *ListNode, k int) *ListNode {
    nodek := head
    for {
        if k == 0 {
            break
        }
        head = head.Next
        k -= 1
    }
    for {
        if head == nil {
            break
        }
        head = head.Next
        nodek = nodek.Next
    }
    return nodek
}
```

查找单链表的中间结点

```
func getMiddleNode(head *ListNode, k int) *ListNode {
    if head == nil || head.Next == nil {
        return head
    }

    faster := head
    for {
        if faster == nil || faster.Next == nil {
            break
        }
        head = head.Next
        faster = faster.Next
        if faster !=nil && faster.Next != nil {
            faster = faster.Next
        }
    }
    return head
}
```

[从尾到头打印单链表]

```go
func reversePrint(head *ListNode) []int {
    var arr, result []int
    if head == nil {
        return result
    }

    for {
        now := head.Val
        arr = append(arr, now)
        head = head.Next
        if head == nil {
            break
        }
    }

    for i:=len(arr)-1; i>=0; i-- {
        result = append(result, arr[i])
    }
    return result
}
```

合并两个排序的链表

```go
func mergeTwoLists(l1 *ListNode, l2 *ListNode) *ListNode {
    if l1 == nil {
        return l2
    }
    if l2 == nil {
        return l1
    }
    var result *ListNode
    if l1.Val < l2.Val {
        result = l1
        result.Next = mergeTwoLists(l1.Next, l2)
    } else {
        result = l2
        result.Next = mergeTwoLists(l1, l2.Next)
    }
    return result
}
```

判断一个单链表中是否有环

```go
func hasCycle(head *ListNode) bool {
    if head == nil {
        return false
    }

    faster := head
    for {
        if faster == nil || faster.Next == nil {
            return false
        }
        faster = faster.Next.Next
        if head.Next == nil {
            return false
```

```
        }
        head = head.Next
        if faster == head {
            return true
        }
    }
    return false
}
```

两个单链表相交的第一个节点

```
func getIntersectionNode(headA, headB *ListNode) *ListNode {
    if headA == nil || headB == nil {
        return nil
    }

    a, b := headA, headB
    len_a, len_b := 1, 1
    for {
        if a.Next == nil {
            break
        }
        len_a += 1
        a = a.Next
    }
    for {
        if b.Next == nil {
            break
        }
        len_b += 1
        b = b.Next
    }
    if a != b {
        return nil
    }

    if len_a > len_b {
        count := len_a - len_b
        for count > 0 {
            count -= 1
            headA = headA.Next
        }
    } else if len_a < len_b {
         count := len_b - len_a
        for count > 0 {
            count -= 1
            headB = headB.Next
        }
    }

    for {
        if headA == headB {
            return headA
        }
        headA = headA.Next
        headB = headB.Next
    }
```

```
    return nil
}
```

已知单链表中存在环，求进入环中第一个节点

```
func detectCycle(head *ListNode) *ListNode {
    if head == nil {
        return nil
    }

    origin := head

    faster := head
    for {
        if faster == nil || faster.Next == nil {
            return nil
        }
        faster = faster.Next.Next
        if head.Next == nil {
            return nil
        }
        head = head.Next
        if faster == head {
            break
        }
    }

    headA, headB := origin, faster

    a, b := headA, headB
    len_a, len_b := 1, 1
    for {
        if a.Next == headB {
            break
        }
        len_a += 1
        a = a.Next
    }
    for {
        if b.Next == headB {
            break
        }
        len_b += 1
        b = b.Next
    }


    if len_a > len_b {
        count := len_a - len_b
        for count > 0 {
            count -= 1
            headA = headA.Next
        }
    } else if len_a < len_b {
         count := len_b - len_a
```

```
        for count > 0 {
            count -= 1
            headB = headB.Next
        }
    }

    for {
        if headA == headB {
            return headA
        }
        headA = headA.Next
        headB = headB.Next
    }

    return nil
}
```

[删除链表的节点]

```
func deleteNode(head *ListNode, val int) *ListNode {
    if head == nil {
        return head
    }
    if head.Val == val {
        return head.Next
    }
    if head.Next != nil && head.Next.Val == val {
        head.Next = head.Next.Next
    }

    deleteNode(head.Next, val)

    return head
}
```

3.2 队列

3.3 栈

3.4 堆

3.5 二叉树

二叉树结构

```
// Definition for a binary tree node.
type TreeNode struct {
    Val int
    Left *TreeNode
    Right *TreeNode
 }
```

[二叉搜索树的最近公共祖先]

```
struct TreeNode* lowestCommonAncestor(struct TreeNode* root, struct TreeNode* p,
struct TreeNode* q) {
    if (root == NULL || p == root || q == root) return root;

    if ((p->val < root->val && q->val > root->val) || (p->val > root->val && q-
>val < root->val))
        return root;

    if (p->val < root->val && q->val < root->val)
        return lowestCommonAncestor(root->left, p, q);

    if (p->val > root->val && q->val > root->val)
        return lowestCommonAncestor(root->right, p, q);

    return root;
}
```

二叉树的直径

二叉树的最大距离（即相距最远的两个叶子节点）

[二叉树的深度]

```
func maxDepth(root *TreeNode) int {
    result := 0
    if root == nil {
        return result
    }

    stack := []*TreeNode{}
    stack = append(stack, root)

    for {
        if len(stack) == 0 {
            break
        }
        line := []int{}
        now_stack := []*TreeNode{}

        for i := 0;  i < len(stack); i++ {
                line = append(line, stack[i].Val)
                if stack[i].Left != nil {
                    now_stack = append(now_stack, stack[i].Left)
                }
                if stack[i].Right != nil {
                    now_stack = append(now_stack, stack[i].Right)
                }
            }
        stack = now_stack
        result = result + 1
    }

    return result
}
```

二叉树的宽度

3.6 并查集

3.7 树状数组

3.8 线段树

3.9 Trie树

3.10 散列表

3.11 二叉排序树

第四章 动态规划

4.1 01背包问题

```
package main
import "fmt"

type Bag struct {
    volume int
    value int
}

func max(a, b int) int {
    if a > b {
        return a
    }
    return b
}

func max_value(bags []*Bag ,capacity int) int {
    n := len(bags)
    var dp [100][100]int

    for i:= 1 ; i<n; i++ {
        for j:=0; j<=capacity; j++ {
            if j >= bags[i].volume {
                dp[i][j] = max(dp[i-1][j],dp[i-1][j-bags[i].volume] +
bags[i].value)
            } else {
                dp[i][j] = dp[i-1][j]
            }
        }
    }

    return dp[n-1][capacity]
}

func main () {
    bag1 := &Bag{1, 2}
    bag2 := &Bag{3, 4}
    bag3 := &Bag{2, 3}
```

```
    bags := []*Bag{bag1, bag2, bag3}
    capacity := 5
    fmt.Println(max_value(bags, capacity))
}
```

4.2 最长递增子序列

```
package main
import "fmt"

func max(a, b int) int {
    if a > b {
        return a
    }
    return b
}

func lis(a[]int) int {
    var dp [101]int
    n := len(a)

    for i:=0; i<n; i++ {
        dp[i] = 1
        for j:=0; j<i; j++ {
            if a[i] > a[j] {
                dp[i] = max(dp[i], dp[j]+1)
            }
        }
    }
    return dp[n-1]
}

func main () {
    a := []int{1, 3, 2, 5, 3, 7, 4, 5}
    fmt.Println(lis(a))
}
```

4.3 最长公共子序列

```
package main
import "fmt"

func max(a, b int) int {
    if a > b {
        return a
    }
    return b
}

func lcs(a, b []int) int {
    var dp [101][101]int
    n, m := len(a), len(b)

    for i:=1; i<=len(a); i++ {
        for j:=1; j<=len(b); j++ {
```

```
            if a[i-1] == b[j-1] {
                dp[i][j] = max(max(dp[i-1][j-1] + 1, dp[i-1][j]), dp[i][j-1])
            } else {
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
            }
        }
    }
    return dp[n][m]
}

func main () {
    a := []int{1, 2, 3, 4, 5}
    b := []int{3, 4, 4, 4, 5}
    fmt.Println(lcs(a, b))
}
```

4.4 最大字段和

```
func maxSubArray(nums []int) int {
    sum, res := nums[0], nums[0]

    for i := 1; i < len(nums); i++ {
        if sum > 0 {
            sum += nums[i]
        } else {
            if nums[i] > sum {
                sum = nums[i]
            }
        }
        if sum > res {
            res = sum
        }
    }
    return res
}
```

4.5 树塔问题

```
func minimumTotal(triangle [][]int) int {
    n := len(triangle)
    var dp [210][210]int

    for i:=n-1; i>=0; i-- {
        for j:=0; j<=i; j++ {
            if triangle[i][j] + dp[i+1][j] < triangle[i][j] + dp[i+1][j+1] {
                dp[i][j] = triangle[i][j] + dp[i+1][j]
            } else {
                dp[i][j] = triangle[i][j] + dp[i+1][j+1]
            }
        }
    }
    return dp[0][0]
}
```

4.6 两个字符串的编辑距离

第五章 搜索

5.1 二分查找

```go
package main
import "fmt"

func binarySearchIndex(arr []int, x int) int {
    st, ed := 0, len(arr)-1
    for st <= ed {
        mid := (st + ed)/2
        if arr[mid] == x {
            return mid
        }
        if arr[mid] > x {
            ed = mid-1
        } else {
            st = mid+1
        }
    }
    return -1
}

func main () {
    var arr = []int{1,2,3,4,6,7,8,9,10}
    x := 5
    fmt.Println(binarySearchIndex(arr, x))
}
```

5.2 BFS

5.3 DFS

5.4 A*

5.5 三分

第六章 图算法

6.1 最短路径

6.2 二分图

6.3 拓扑排序

6.4 最小生成树

第七章 字符串算法

7.1 KMP

7.2 BM

7.3 Manacher

7.4 AC自动机

第八章 海量数据处理

8.1 布隆过滤器

8.2 Bitmap

8.3 Simhash

8.4 MapReduce